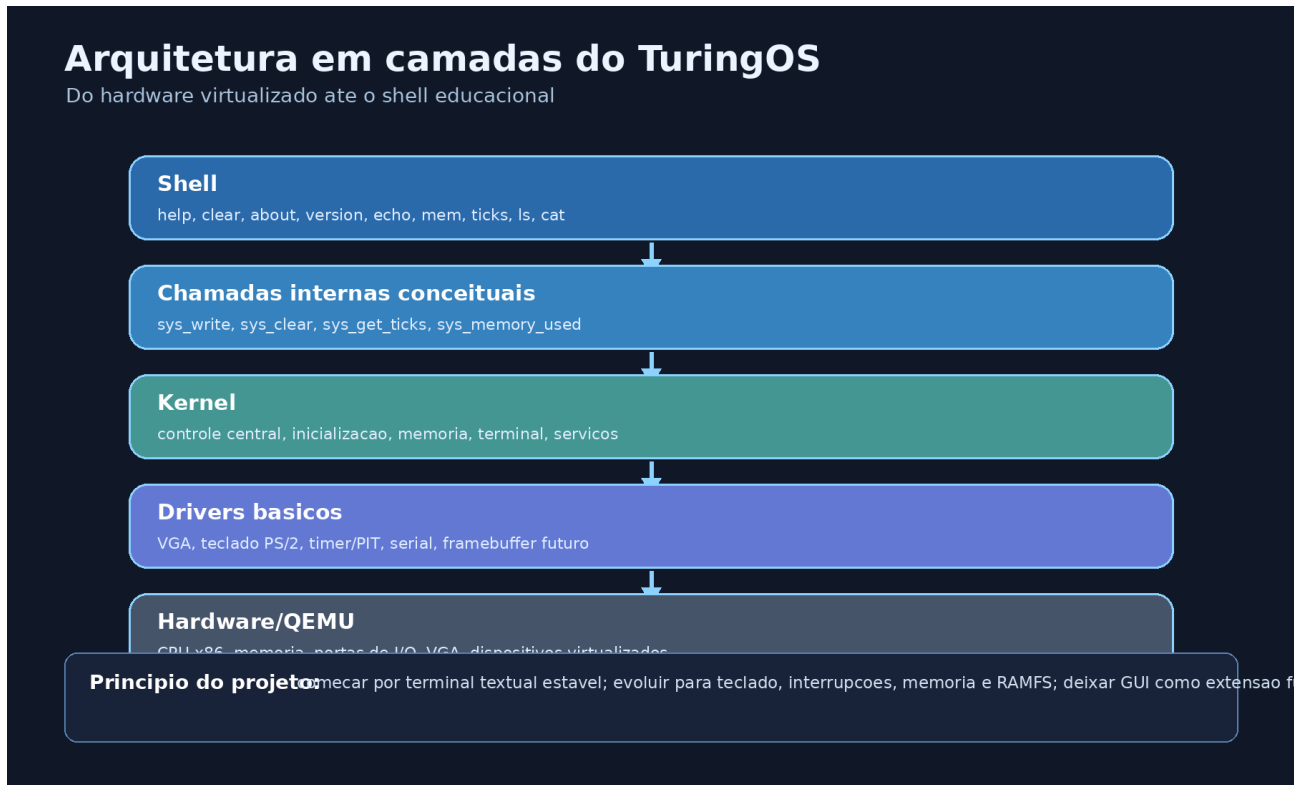


TuringOS

Documentacao completa do projeto

Do planejamento inicial a versao final de apresentacao



Autor: Joseph Antony

Ano: 2026

Projeto academico de Interface Hardware/Software

Resumo: Este documento organiza todo o desenvolvimento do TuringOS, incluindo objetivo, escopo, arquitetura, ambiente, fases, guia de implementacao, testes, depuracao, engenharia reversa, melhorias por etapa e evolucoes futuras.

Sumario

1. 1. Visao geral do projeto
2. 2. Objetivos, escopo e criterios de sucesso
3. 3. Aderencia aos requisitos da disciplina
4. 4. Ambiente de desenvolvimento e instalacao das ferramentas
5. 5. Arquitetura geral do TuringOS
6. 6. Estrutura do repositorio e convencoes
7. 7. GitHub Projects, issues, milestones e workflow Git
8. 8. Roadmap de versoes
9. 9. Plano de execucao completo por etapas
10. 10. Guia de implementacao inicial
11. 11. Testes, depuracao e prototipacao virtual
12. 12. Engenharia reversa sem codigo-fonte
13. 13. Documentacao, apresentacao e demonstracao final
14. 14. Riscos, contingencias e criterios de corte
15. 15. Melhorias por etapa
16. 16. Melhorias futuras e evolucao avancada
17. 17. Checklists finais
18. 18. Glossario
19. 19. Referencias recomendadas

1. Visao geral do projeto

O TuringOS e um mini sistema operacional educacional inspirado em Unix. A proposta e construir um kernel didatico, pequeno e verificavel, capaz de inicializar em uma maquina virtual, executar codigo em C chamado por Assembly, escrever diretamente na tela e evoluir para terminal, teclado, shell, interrupcoes, timer, memoria simples e sistema de arquivos em RAM.

O projeto nao tem o objetivo de competir com Linux, BSD, Minix ou sistemas operacionais completos. O foco e pedagogico: entender a interface entre hardware e software, o processo de boot, a geracao de binarios, a escrita direta em memoria de video, o uso de portas de entrada e saida, a depuracao de baixo nivel e a organizacao do desenvolvimento com Git e Kanban.

Elemento	Descricao
Nome	TuringOS
Tipo	Mini sistema operacional educacional
Linguagens	C e Assembly x86
Execucao	QEMU, preferencialmente via WSL2 Ubuntu
Boot	GRUB com cabecalho Multiboot
Interface inicial	Terminal textual em modo VGA
Evolucao	Teclado, shell, interrupcoes, timer, memoria, RAMFS e GUI experimental futura

Ideia central: Entregar primeiro um kernel minimo que inicializa e escreve na tela; depois evoluir por camadas, sempre mantendo o projeto compilavel, testavel e documentado.

2. Objetivos, escopo e criterios de sucesso

2.1 Objetivo geral

Desenvolver um mini kernel educacional em C e Assembly para demonstrar, na pratica, conceitos de Interface Hardware/Software, Sistemas Operacionais, Arquitetura de Computadores e programacao de baixo nivel.

2.2 Objetivos especificos

- Configurar ambiente de desenvolvimento em WSL2 Ubuntu ou Linux nativo.
- Criar estrutura de projeto com codigo, documentacao, scripts, imagens e artefatos de build.
- Escrever codigo Assembly de entrada com cabecalho Multiboot e chamada ao kernel em C.
- Criar um kernel minimo em C, sem depender da biblioteca padrao de um sistema operacional hospedeiro.
- Gerar binario executavel e imagem inicializavel via linker script, GRUB e xorriso.
- Executar o sistema em QEMU e validar o comportamento do kernel.
- Implementar saida de texto por acesso direto a memoria VGA em 0xB8000.
- Criar terminal textual basico com limpeza de tela, quebra de linha e prompt.
- Adicionar, se houver tempo, teclado, shell, interrupcoes, timer, memoria simples e RAMFS.
- Documentar depuracao com QEMU/GDB e engenharia reversa de binario sem codigo-fonte.
- Organizar o desenvolvimento com Git, issues, milestones e GitHub Projects/Kanban.

2.3 Escopo minimo

- Boot com GRUB

- Assembly inicial
- Kernel em C
- linker.ld
- Makefile
- QEMU
- VGA modo texto
- Terminal basico
- Documentacao de debug
- Engenharia reversa sem codigo-fonte
- Git e Kanban

2.4 Escopo recomendado

- Entrada por teclado
- Buffer de comandos
- Shell simples
- Comandos help, clear, about, version e echo
- Porta serial para logs
- Organizacao modular do codigo

2.5 Escopo avancado

- IDT, ISR, IRQ e PIC
- Timer/PIT
- kmalloc simples
- RAMFS
- Chamadas internas conceituais
- Interface textual avancada
- Framebuffer grafico experimental
- Multitarefa cooperativa futura

2.6 Criterios de sucesso

Criterio	Como validar
Kernel carrega	QEMU inicia e GRUB transfere controle para o kernel
Assembly integrado	boot.asm configura pilha e chama kernel_main
Binario gerado	build/turingos.bin ou ISO e criado pelo Makefile
VGA funciona	Mensagem aparece na tela sem printf
Projeto reproduzivel	Outro usuario consegue rodar make run
Depuracao documentada	docs/debugging.md mostra QEMU/GDB/objdump
Engenharia reversa documentada	docs/reverse-engineering.md analisa binario sem codigo-fonte
Kanban demonstravel	GitHub Projects mostra tarefas por semana e status

3. Aderencia aos requisitos da disciplina

A tabela abaixo conecta os requisitos academicos do projeto com as entregas planejadas no TuringOS.

Requisito	Atendimento no TuringOS	Evidencia
Interface direta com hardware	Escrita em VGA 0xB8000, portas de I/O, teclado, timer e serial	vga.c, keyboard.c, timer.c, serial.c
Assembly	Ponto de entrada, Multiboot, pilha inicial e	boot/boot.asm

	chamada para C	
Geracao de binario	NASM/GCC/LD gerando objetos, kernel e ISO	Makefile, linker.ld, build/
Gerenciamento de dispositivo	Drivers proprios para VGA, teclado, timer e serial	kernel/drivers
Engenharia reversa sem fonte	Analise de binario com hexdump, objdump, ndisasm e GDB	docs/reverse-engineering.md
Depuracao/prototipacao virtual	Execucao em QEMU, GDB remoto, logs serial, objdump	docs/debugging.md
Kanban semanal	Issues por semana, milestones, status e GitHub Projects	GitHub Project
Versionamento	Branches, commits semanticos, tags e historico	Git/GitHub

Observacao importante: A engenharia reversa deve ser descrita como analise de binario sem acesso ao codigo-fonte. O estudo de xv6, Minix e OSDev deve aparecer como referencia conceitual complementar, nao como substituto dessa etapa.

4. Ambiente de desenvolvimento e instalacao das ferramentas

O ambiente recomendado e Windows 11 com WSL2 Ubuntu. A compilacao e execucao do kernel devem ocorrer no Ubuntu/WSL, porque as ferramentas de OSDev, GRUB, QEMU e binutils funcionam de maneira mais natural no Linux.

```
# No PowerShell do Windows
wsl -l -v
wsl
```

4.1 Instalacao completa no Ubuntu/WSL

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y build-essential nasm make gcc-multilib binutils qemu-system-x86 qemu-system-gui qemu-utils grub-pc-bin grub-common xorriso mtools gdb git
```

4.2 Verificacao das ferramentas

```
gcc --version
make --version
nasm -v
ld --version
objdump --version
readelf --version
qemu-system-i386 --version
grub-mkrescue --version
xorriso --version
gdb --version
git --version
```

4.3 Configuracao do Git

```
git config --global user.name "Joseph Antony"
git config --global user.email "joseph_adsl@academico.ufs.br"
git config --global --list
```

4.4 Clonagem do repositório

```
cd ~
git clone https://github.com/thejosephantony/TuringOS.git
cd TuringOS
```

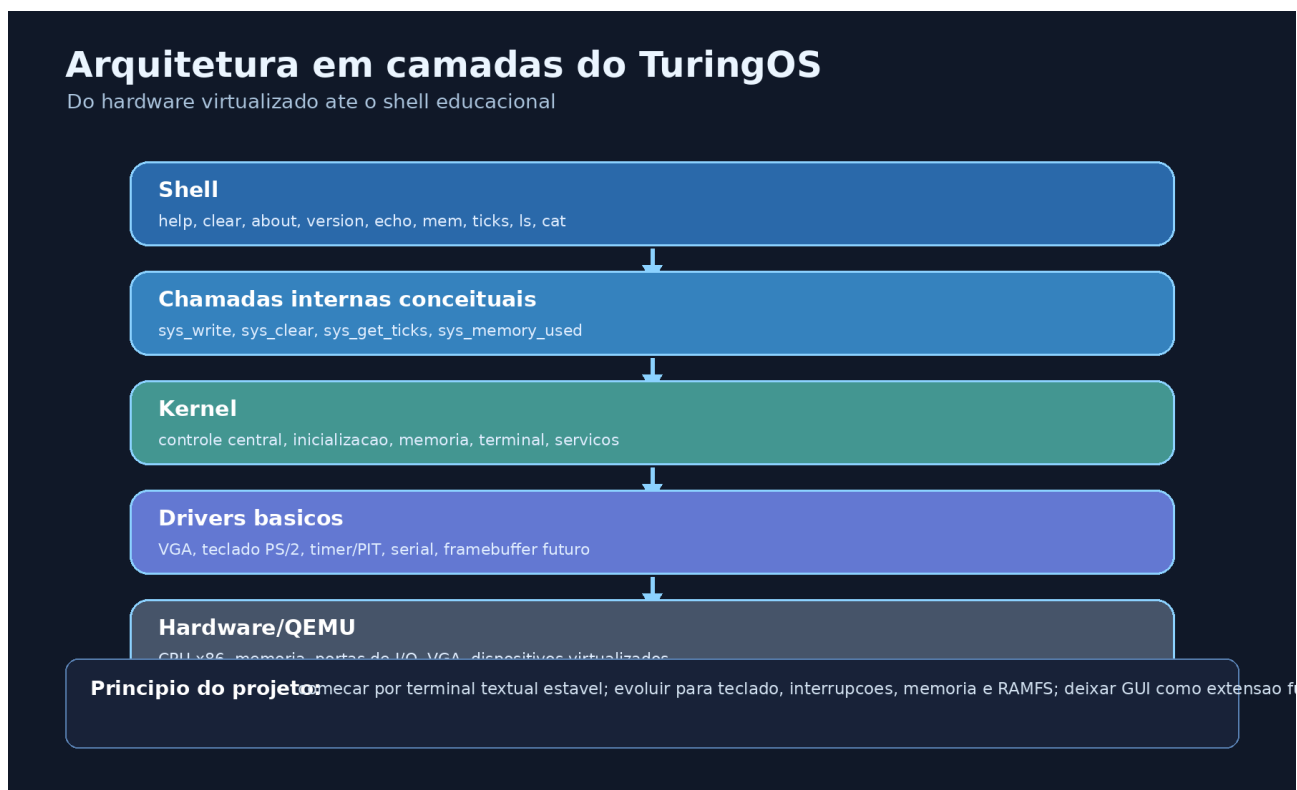
4.5 VS Code com WSL

Instale o Visual Studio Code no Windows e a extensão WSL. Depois, dentro do Ubuntu/WSL:

```
cd ~/TuringOS
code .
```

Ferramenta	Funcao no projeto
GCC	Compilar C em modo freestanding
NASM	Montar Assembly x86
LD/binutils	Linkar e inspecionar binarios
GRUB	Carregar kernel via Multiboot
xorriso	Gerar ISO inicializavel
QEMU	Executar o kernel em maquina virtual
GDB	Depurar kernel
Make	Automatizar build
Git	Versionar codigo
GitHub CLI	Automatizar issues, milestones e Project

5. Arquitetura geral do TuringOS



A arquitetura inicial e organizada em camadas. A camada mais baixa e o hardware virtualizado pelo QEMU. Acima dela ficam os drivers que conversam diretamente com memoria e portas de I/O. O kernel coordena os drivers e expõe servicos internos. O terminal organiza a saida textual. O shell interpreta comandos do usuario.

5.1 Fluxo de boot

BIOS/QEMU

-> GRUB

-> boot/boot.asm

-> configura pilha

-> chama kernel_main()

-> inicializa terminal

-> inicializa drivers

-> mostra prompt

5.2 Principios arquiteturais

- Simplicidade antes de completude: cada fase deve ter um resultado executavel.
- Camadas pequenas: shell nao deve escrever diretamente em VGA; deve usar terminal ou chamadas internas.
- Baixo acoplamento: driver VGA, terminal, teclado, timer e memoria devem ficar em arquivos separados.
- Documentacao junto com codigo: cada recurso importante deve ter explicacao em docs/.
- Testabilidade manual: cada etapa deve ser demonstravel no QEMU.
- Evolucao controlada: GUI, multitarefa e paginacao ficam como futuras extensoes, nao como base inicial.

6. Estrutura do repositorio e convencoes

6.1 Estrutura planejada

TuringOS/

├─ boot/

| └─ boot.asm

├─ kernel/

| └─ kernel.c

| └─ vga.c

| └─ terminal.c

| └─ keyboard.c

| └─ timer.c

| └─ memory.c

| └─ ramfs.c

| └─ serial.c

| └─ shell.c

├─ include/

| └─ vga.h

| └─ terminal.h

| └─ keyboard.h

| └─ timer.h

| └─ memory.h

| └─ ramfs.h

| └─ serial.h

| └─ shell.h

└─ iso/

```

|   └─ boot/
|       └─ grub/
|           └─ grub.cfg
└─ docs/
    └─ images/
        └─ debugging.md
        └─ reverse-engineering.md
        └─ kanban.md
        └─ architecture.md
        └─ execution-plan.md
└─ build/
    └─ linker.ld
    └─ Makefile
    └─ LICENSE
    └─ README.md

```

6.2 Convencoes de nomes

Item	Convencao
Arquivos C	snake_case.c, por modulo
Headers	mesmo nome do modulo em include/
Funcoes	prefixo do modulo, ex.: vga_write, terminal_clear
Branches	feature/nome-do-modulo, docs/nome-do-doc
Commits	feat:, fix:, docs:, chore:, build:
Tags	v0.1, v0.2, v0.3, v1.0

7. GitHub Projects, issues, milestones e workflow Git

O GitHub Projects deve funcionar como o quadro Kanban do desenvolvimento. As issues representam tarefas, as milestones representam entregas por versao e as labels classificam tipo, prioridade e modulo.

7.1 Colunas sugeridas

Coluna	Uso
Backlog	Ideias futuras e extras
A fazer	Tarefas planejadas da semana
Em desenvolvimento	Tarefa ativa
Em teste	Implementado, mas ainda validando
Concluido	Finalizado, commitado e documentado

7.2 Milestones

Milestone	Conteudo
v0.1 - Boot e kernel minimo	estrutura, ambiente, boot.asm, kernel.c, linker, GRUB, Makefile, QEMU
v0.2 - VGA e terminal	driver VGA, escrita em 0xB8000, clear, terminal
v0.3 - Debug e engenharia reversa	QEMU/GDB, objdump, hexdump, ndisasm, docs
v1.0 - Entrega final	documentacao, slides, demo, revisao final

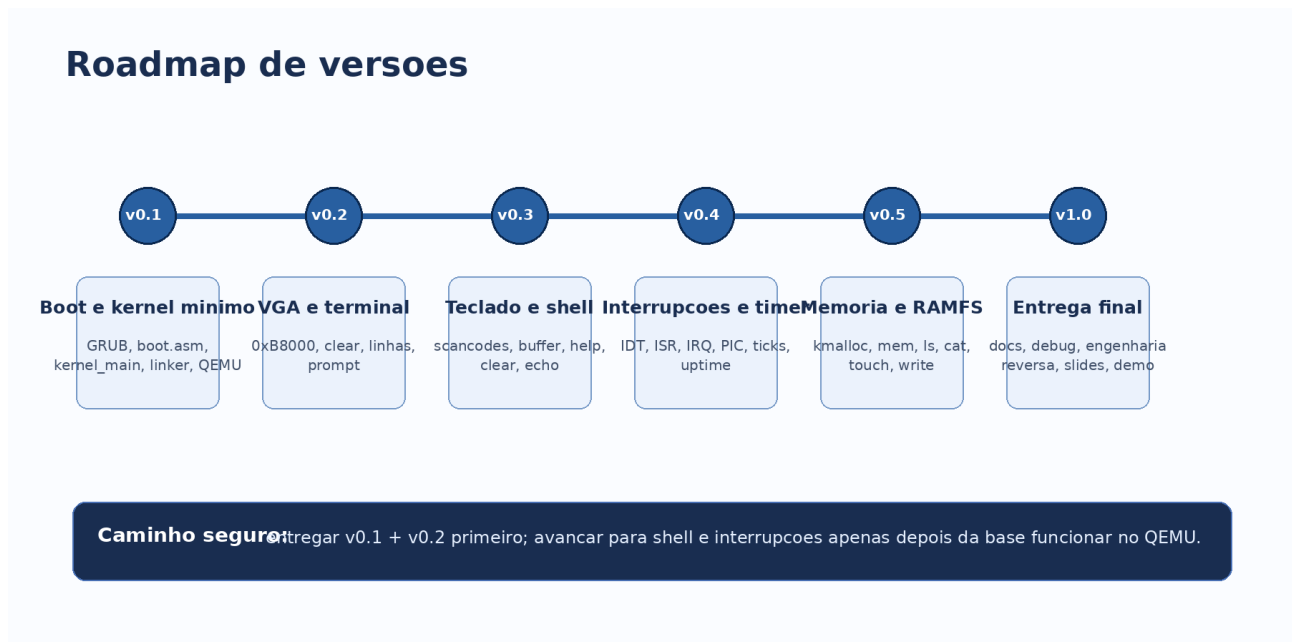
7.3 Workflow Git

```
git checkout -b dev
git push -u origin dev
```

```
git checkout -b feature/project-structure
# implementar a tarefa
git add .
git commit -m "chore: create initial project structure"
git push -u origin feature/project-structure
```

Regra: Cada issue deve ter pelo menos um commit relacionado. Ao concluir uma tarefa, fechar a issue com comentario indicando o que foi entregue.

8. Roadmap de versoes



Versao	Objetivo	Resultado esperado
v0.1	Boot e kernel minimo	Kernel inicializa no QEMU
v0.2	VGA e terminal	Texto e prompt aparecem na tela
v0.3	Teclado e shell	Usuario digita comandos basicos
v0.4	Interrupcoes e timer	Ticks e uptime funcionam
v0.5	Memoria e RAMFS	Comandos mem, ls, cat, touch e write
v1.0	Entrega final	Projeto documentado e demonstravel

9. Plano de execucao completo por etapas

Fluxo de desenvolvimento e demonstracao



- Cada etapa deve gerar um commit pequeno e verificavel.

- Toda funcao de baixo nivel deve ter teste manual no QEMU.

- A documentacao acompanha o codigo: nao deixar para escrever apenas no final.

Fase 0 - Planejamento e repositorio

Definir escopo, README, GitHub Project, labels, milestones e issues.

Atividades	Entregaveis	Melhorias possiveis
- Criar/atualizar README - Criar Project/Kanban - Criar issues por semana - Definir v0.1, v0.2, v0.3 e v1.0	- Repositorio publico organizado - Issues e milestones cadastradas - Kanban visivel	- Adicionar templates de issue - Criar CONTRIBUTING.md - Criar checklist de avaliacao no README

Fase 1 - Ambiente de desenvolvimento

Preparar WSL2 Ubuntu com todas as ferramentas.

Atividades	Entregaveis	Melhorias possiveis
- Instalar GCC, NASM, Make, QEMU, GRUB, xorriso, GDB - Configurar Git - Clonar repositorio - Testar qemu-system-i386	- Ambiente reproduzivel - Comandos de verificacao funcionando	- Criar script setup.sh - Adicionar devcontainer futuramente - Documentar erros comuns de WSL/QEMU

Fase 2 - Estrutura inicial

Criar a organizacao base de pastas e arquivos.

Atividades	Entregaveis	Melhorias possiveis
- Criar boot/ - Criar kernel/ - Criar include/ - Criar iso/boot/grub/ - Criar docs/ - Criar Makefile vazio	- Estrutura base versionada	- Adicionar .editorconfig - Adicionar Makefile com help - Separar build/ de codigo-fonte

Fase 3 - Boot com GRUB e Assembly

Fazer o GRUB carregar o kernel e transferir controle para Assembly.

Atividades	Entregaveis	Melhorias possiveis
- Criar cabeçalho Multiboot - Definir entry point - Configurar stack - Chamar kernel_main	- boot.asm montado - GRUB reconhece o kernel	- Adicionar comentarios detalhados no Assembly - Gerar dump do objeto boot.o - Documentar registradores usados

Fase 4 - Kernel minimo em C

Criar kernel_main sem depender da biblioteca padrao.

Atividades	Entregaveis	Melhorias possiveis
- Criar kernel/kernel.c - Evitar printf, malloc, fopen - Criar loop final de seguranca - Integrar com boot.asm	- Kernel compila e executa	- Criar tipos basicos em include/types.h - Adicionar panic() - Adicionar versao do kernel

Fase 5 - Linker script e binario

Definir layout de memoria do kernel.

Atividades	Entregaveis	Melhorias possiveis
- Criar linker.ld - Definir ENTRY - Definir secoes .text, .rodata, .data, .bss - Gerar turingos.bin	- Binario gerado em build/	- Gerar map file - Documentar enderecos de memoria - Comparar readelf/objdump

Fase 6 - GRUB, ISO e Makefile

Automatizar build, ISO, run, debug e clean.

Atividades	Entregaveis	Melhorias possiveis
- Criar grub.cfg - Criar targets make, iso, run, debug, clean - Gerar ISO - Executar QEMU	- make run funciona	- Adicionar make help - Adicionar make disasm - Adicionar make debug-serial - Adicionar validacao de dependencias

Fase 7 - Driver VGA

Escrever diretamente em 0xB8000.

Atividades	Entregaveis	Melhorias possiveis
- Criar vga.c/h - Implementar putchar - Implementar write - Implementar clear - Controlar linha e coluna	- Mensagem aparece no QEMU	- Adicionar cores - Adicionar scroll - Adicionar cursor visual - Adicionar suporte a tabs e backspace

Fase 8 - Terminal basico

Criar camada de terminal sobre VGA.

Atividades	Entregaveis	Melhorias possiveis
- terminal_initialize - terminal_write - terminal_writeln - terminal_clear - Prompt inicial	- Terminal organizado	- Criar tema visual - Criar painel textual - Adicionar historico de comandos futuramente

Fase 9 - Porta serial para logs

Adicionar saida de debug independente do VGA.

Atividades	Entregaveis	Melhorias possiveis
- Inicializar COM1 - serial_write - serial_writeln - Logs de inicializacao	- Logs aparecem no terminal do QEMU	- Criar niveis INFO/WARN/ERROR - Enviar panic para serial - Criar make run-serial

Fase 10 - Teclado por polling

Permitir entrada basica sem interrupcoes inicialmente.

Atividades	Entregaveis	Melhorias possiveis
- Ler porta 0x60 - Converter scancode para ASCII - Tratar Enter e Backspace - Criar buffer	- Usuario digita texto	- Adicionar Shift - Adicionar layout ABNT2 futuramente - Migrar para interrupcao depois

Fase 11 - Shell simples

Interpretar comandos internos.

Atividades	Entregaveis	Melhorias possiveis
- Criar shell.c/h - Criar prompt - Implementar help, clear, about, version, echo	- Shell funcional	- Adicionar parse de argumentos - Adicionar comando history - Adicionar comandos mem/ticks/ls futuramente

Fase 12 - Interrupcoes

Criar IDT, ISRs, IRQs e configurar PIC.

Atividades	Entregaveis	Melhorias possiveis
- Criar idt.c - Criar isr.c - Criar irq.c - Criar stubs Assembly - Habilitar interrupcoes	- Excecoes basicas tratadas	- Criar mensagens para excecoes - Registrar handlers dinamicos - Documentar vetor por vetor

Fase 13 - Timer/PIT

Contar ticks e uptime.

Atividades	Entregaveis	Melhorias possiveis
- Configurar PIT	- Sistema conta tempo	- Adicionar sleep

- Incrementar contador - Criar timer_get_ticks - Criar comandos ticks e uptime		- Base para escalonador - Medir tempo de comandos
--	--	--

Fase 14 - Memoria simples

Criar alocador linear kmalloc.

Atividades	Entregaveis	Melhorias possiveis
- Definir heap_start - Criar kmalloc - Contabilizar memoria usada - Criar comando mem	- Alocacao basica funciona	- Adicionar alinhamento - Adicionar kfree futuramente - Adicionar mapa de memoria do Multiboot

Fase 15 - RAMFS

Criar sistema de arquivos em memoria.

Atividades	Entregaveis	Melhorias possiveis
- Criar struct file - Criar touch - Criar write - Criar ls - Criar cat	- Arquivos temporarios em RAM	- Adicionar diretorios - Adicionar limite dinamico com kmalloc - Persistencia em disco futuramente

Fase 16 - Chamadas internas conceituais

Separar shell dos servicos do kernel.

Atividades	Entregaveis	Melhorias possiveis
- sys_write - sys_clear - sys_get_ticks - sys_memory_used	- Arquitetura em camadas	- Transformar em syscalls reais via int 0x80 futuramente - Criar tabela de chamadas

Fase 17 - Interface textual avancada

Melhorar apresentacao visual sem GUI real.

Atividades	Entregaveis	Melhorias possiveis
- Criar caixas em modo texto - Criar tela de status - Criar menu de comandos - Usar cores	- Apresentacao mais profissional	- Criar dashboard TUI - Criar pseudo-janelas em texto - Adicionar tema escuro/claro

Fase 18 - Documentacao e apresentacao final

Fechar README, docs, prints, slides e demonstracao.

Atividades	Entregaveis	Melhorias possiveis
- Atualizar README - Criar docs/debugging.md - Criar docs/reverse-engineering.md - Tirar prints - Preparar pitch	- Projeto pronto para entrega	- Adicionar GIF - Criar release v1.0 - Adicionar roteiro de demo em video

10. Guia de implementacao inicial

Esta secao apresenta um guia inicial de implementacao. Os codigos sao esqueletos de referencia e devem ser ajustados durante o desenvolvimento.

10.1 boot/boot.asm

```
; boot/boot.asm - entrada inicial do TuringOS
BITS 32
```

```
MBALIGN equ 1 << 0
MEMINFO equ 1 << 1
FLAGS equ MBALIGN | MEMINFO
MAGIC equ 0x1BADB002
CHECKSUM equ -(MAGIC + FLAGS)
```

```
section .multiboot
align 4
    dd MAGIC
    dd FLAGS
    dd CHECKSUM
```

```
section .bss
align 16
stack_bottom:
    resb 16384
stack_top:
```

```
section .text
global _start
extern kernel_main
```

```
_start:
    mov esp, stack_top
    call kernel_main
```

```
.hang:
    cli
    hlt
    jmp .hang
```

10.2 kernel/kernel.c

```
// kernel/kernel.c
#include "terminal.h"
```

```
void kernel_main(void) {
    terminal_initialize();
    terminal_writeln("TuringOS v0.1");
    terminal_writeln("Kernel loaded successfully.");
}
```

```

terminal_writeln("An educational Unix-like operating system.");
terminal_write("\nturingos> ");

while (1) {
    __asm__ volatile ("hlt");
}
}

```

10.3 linker.ld

```

ENTRY(_start)

SECTIONS
{
    . = 1M;

    .text BLOCK(4K) : ALIGN(4K) {
        *(.multiboot)
        *(.text)
    }

    .rodata BLOCK(4K) : ALIGN(4K) {
        *(.rodata)
    }

    .data BLOCK(4K) : ALIGN(4K) {
        *(.data)
    }

    .bss BLOCK(4K) : ALIGN(4K) {
        *(COMMON)
        *(.bss)
    }
}

```

10.4 iso/boot/grub/grub.cfg

```

set timeout=0
set default=0

menuentry "TuringOS" {
    multiboot /boot/turingos.bin
    boot
}

```

10.5 Makefile inicial

```

AS=nasm
CC=gcc
LD=ld
CFLAGS=-m32 -ffreestanding -fno-stack-protector -fno-pic -Wall -Wextra -linclude
LDFLAGS=-m elf_i386 -T linker.ld

```

```

BUILD=build
ISO_DIR=iso
KERNEL=$(BUILD)/turingos.bin
ISO=$(BUILD)/turingos.iso

OBJS=$(BUILD)/boot.o $(BUILD)/kernel.o $(BUILD)/vga.o $(BUILD)/terminal.o

all: $(KERNEL)

$(BUILD):
    mkdir -p $(BUILD)

$(BUILD)/boot.o: boot/boot.asm | $(BUILD)
    $(AS) -f elf32 $< -o $@

$(BUILD)/kernel.o: kernel/kernel.c | $(BUILD)
    $(CC) $(CFLAGS) -c $< -o $@

$(BUILD)/vga.o: kernel/vga.c | $(BUILD)
    $(CC) $(CFLAGS) -c $< -o $@

$(BUILD)/terminal.o: kernel/terminal.c | $(BUILD)
    $(CC) $(CFLAGS) -c $< -o $@

$(KERNEL): $(OBJS)
    $(LD) $(LDFLAGS) -o $@ $(OBJS)

iso: $(KERNEL)
    mkdir -p $(ISO_DIR)/boot/grub
    cp $(KERNEL) $(ISO_DIR)/boot/turingos.bin
    grub-mkrescue -o $(ISO) $(ISO_DIR)

run: iso
    qemu-system-i386 -cdrom $(ISO)

debug: $(KERNEL)
    qemu-system-i386 -kernel $(KERNEL) -s -S

clean:
    rm -rf $(BUILD)/*.o $(KERNEL) $(ISO)

.PHONY: all iso run debug clean

```

10.6 Driver VGA conceitual

```

// include/vga.h
#ifndef VGA_H
#define VGA_H

void vga_clear(void);
void vga_putchar(char c);
void vga_write(const char* str);

```



```

void vga_set_color(unsigned char color);

#endif

// kernel/vga.c - ideia inicial
#include "vga.h"

#define VGA_WIDTH 80
#define VGA_HEIGHT 25
#define VGA_MEMORY ((volatile unsigned short*)0xB8000)

static int row = 0;
static int col = 0;
static unsigned char color = 0x07;

static unsigned short vga_entry(char c, unsigned char color) {
    return (unsigned short)c | ((unsigned short)color << 8);
}

void vga_clear(void) {
    for (int y = 0; y < VGA_HEIGHT; y++) {
        for (int x = 0; x < VGA_WIDTH; x++) {
            VGA_MEMORY[y * VGA_WIDTH + x] = vga_entry(' ', color);
        }
    }
    row = 0;
    col = 0;
}

void vga_putchar(char c) {
    if (c == '\n') {
        col = 0;
        row++;
        return;
    }

    VGA_MEMORY[row * VGA_WIDTH + col] = vga_entry(c, color);
    col++;

    if (col >= VGA_WIDTH) {
        col = 0;
        row++;
    }

    if (row >= VGA_HEIGHT) {
        row = 0; // versao simples; melhorar com scroll
    }
}

void vga_write(const char* str) {
    for (int i = 0; str[i] != '\0'; i++) {

```

```

    vga_putchar(str[i]);
}
}

void vga_set_color(unsigned char new_color) {
    color = new_color;
}

```

11. Testes, depuracao e prototipacao virtual

11.1 Execucao normal

```

make clean
make
make iso
make run

```

11.2 Debug com QEMU e GDB

```

# Terminal 1
qemu-system-i386 -kernel build/turingos.bin -s -S

```

```

# Terminal 2
gdb build/turingos.bin
(gdb) target remote localhost:1234
(gdb) break kernel_main
(gdb) continue
(gdb) info registers
(gdb) x/16i $eip

```

11.3 Inspecao de binarios

```

readelf -h build/turingos.bin
readelf -S build/turingos.bin
objdump -d build/turingos.bin | less
hexdump -C build/turingos.bin | head
ndisasm -b 32 build/turingos.bin | less

```

Problema	Causa provavel	Solucao
QEMU abre mas nao mostra nada	Kernel nao foi carregado ou tela nao foi escrita	Verificar GRUB, Multiboot, linker e vga_write
GRUB nao encontra kernel	grub.cfg ou caminho incorreto	Confirmar /boot/turingos.bin dentro da ISO
Erro de arquitetura	Objetos 64-bit misturados com 32-bit	Usar nasm -f elf32, gcc -m32 e ld -m elf_i386
Triple fault/reboot	IDT ausente ou execucao nao tratada	Manter interrupcoes desabilitadas ate configurar IDT
GDB nao conecta	QEMU nao iniciou com -s -S	Rodar qemu-system-i386 -s -S

12. Engenharia reversa sem codigo-fonte

A etapa de engenharia reversa deve analisar um binario sem consultar seu codigo-fonte. A atividade pode usar o proprio turingos.bin gerado ou um pequeno binario de boot/kernel preparado para analise. O objetivo e mostrar que e possivel compreender parte do comportamento por meio de bytes, instrucoes, cabecalhos, enderecos e execucao controlada.

12.1 Roteiro da analise

20. Escolher o binario alvo.
21. Calcular informacoes basicas: tamanho, formato e hash.
22. Visualizar bytes iniciais com hexdump.
23. Identificar cabecalho, ponto de entrada e secoes com readelf, quando aplicavel.
24. Desmontar instrucoes com objdump ou ndisasm.
25. Executar em QEMU/GDB e parar no ponto de entrada.
26. Comparar a analise estatica com o comportamento observado.
27. Registrar conclusoes em docs/reverse-engineering.md.

12.2 Modelo de documento reverse-engineering.md

```
# Engenharia reversa sem codigo-fonte - TuringOS
```

```
## Binario analisado
```

- Arquivo: build/turingos.bin
- Tamanho:
- Hash:

```
## Ferramentas usadas
```

- hexdump
- readelf
- objdump
- ndisasm
- gdb
- qemu-system-i386

```
## Analise estatica
```

- Bytes iniciais:
- Cabecalhos/secoes:
- Ponto de entrada:
- Instrucoes relevantes:

```
## Analise dinamica
```

- Comando QEMU:
- Breakpoint:
- Registradores observados:
- Comportamento visto na tela:

```
## Conclusao
```

```
Explicar o que foi descoberto sem consultar o codigo-fonte.
```

13. Documentacao, apresentacao e demonstracao final

13.1 Documentos minimos

- README.md completo
- docs/architecture.md
- docs/execution-plan.md
- docs/debugging.md
- docs/reverse-engineering.md
- docs/kanban.md
- docs/images com prints do QEMU
- slides de apresentacao

13.2 Roteiro de demonstracao

28. Abrir o repositorio no GitHub.
29. Mostrar README e arquitetura do TuringOS.
30. Mostrar GitHub Projects/Kanban.
31. Mostrar estrutura de pastas.
32. Rodar make clean.
33. Rodar make ou make iso.
34. Rodar make run.
35. Mostrar TuringOS no QEMU.
36. Explicar VGA/terminal ou shell, se implementado.
37. Mostrar docs/debugging.md.
38. Mostrar docs/reverse-engineering.md.
39. Concluir com limitacoes e melhorias futuras.

13.3 Script curto de apresentacao

O TuringOS e um mini sistema operacional educacional inspirado em Unix, desenvolvido em C e Assembly. A proposta e estudar, na pratica, a interface hardware/software: desde o boot pelo GRUB, passando pela entrada em Assembly, ate um kernel em C que escreve diretamente na tela via memoria VGA. O projeto e executado no QEMU, depurado com GDB e documentado com uma etapa de engenharia reversa sem codigo-fonte. O desenvolvimento foi organizado com Git, issues, milestones e Kanban.

14. Riscos, contingencias e criterios de corte

Risco	Impacto	Contingencia
Boot nao funciona	Bloqueia projeto	Usar GRUB/Multiboot e comparar com exemplo minimo conhecido
QEMU/WSL nao abre janela	Atraso em testes	Usar qemu -nographic/serial ou instalar qemu-system-gui/WSLg
Erro 32-bit/64-bit	Linkagem falha	Padronizar elf32, -m32 e ld -m elf_i386
Teclado demora	Shell fica incompleto	Manter teclado como extra e entregar terminal textual
Interrupcoes causam triple fault	Kernel reinicia	Deixar IDT/timer para fase avancada
RAMFS consome tempo	Desvia foco	Deixar como v0.5/futuro
Documentacao atrasada	Perde valor academico	Escrever docs junto com cada commit

14.1 Criterio de corte para 3 semanas

Se o prazo estiver curto, cortar teclado, interrupcoes, timer, memoria e RAMFS. A versao minima deve priorizar boot, kernel, binario, QEMU, VGA, terminal e documentacao de debug/engenharia reversa.

15. Melhorias por etapa

Etapa	Melhorias possiveis
0 - Planejamento e repositorio	- Adicionar templates de issue - Criar CONTRIBUTING.md - Criar checklist de avaliacao no README
1 - Ambiente de desenvolvimento	- Criar script setup.sh - Adicionar devcontainer futuramente - Documentar erros comuns de WSL/QEMU
2 - Estrutura inicial	- Adicionar .editorconfig - Adicionar Makefile com help - Separar build/ de codigo-fonte
3 - Boot com GRUB e Assembly	- Adicionar comentarios detalhados no Assembly - Gerar dump do objeto boot.o - Documentar registradores usados
4 - Kernel minimo em C	- Criar tipos basicos em include/types.h - Adicionar panic() - Adicionar versao do kernel
5 - Linker script e binario	- Gerar map file - Documentar enderecos de memoria - Comparar readelf/objdump
6 - GRUB, ISO e Makefile	- Adicionar make help - Adicionar make disasm - Adicionar make debug-serial - Adicionar validacao de dependencias
7 - Driver VGA	- Adicionar cores - Adicionar scroll - Adicionar cursor visual - Adicionar suporte a tabs e backspace
8 - Terminal basico	- Criar tema visual - Criar painel textual - Adicionar historico de comandos futuramente
9 - Porta serial para logs	- Criar niveis INFO/WARN/ERROR - Enviar panic para serial - Criar make run-serial
10 - Teclado por polling	- Adicionar Shift - Adicionar layout ABNT2 futuramente - Migrar para interrupcao depois
11 - Shell simples	- Adicionar parse de argumentos - Adicionar comando history - Adicionar comandos mem/ticks/ls futuramente
12 - Interrupcoes	- Criar mensagens para excecoes - Registrar handlers dinamicos - Documentar vetor por vetor
13 - Timer/PIT	- Adicionar sleep - Base para escalonador - Medir tempo de comandos
14 - Memoria simples	- Adicionar alinhamento - Adicionar kfree futuramente - Adicionar mapa de memoria do Multiboot
15 - RAMFS	- Adicionar diretorios - Adicionar limite dinamico com kmalloc - Persistencia em disco futuramente
16 - Chamadas internas conceituais	- Transformar em syscalls reais via int 0x80 futuramente

	- Criar tabela de chamadas
17 - Interface textual avancada	- Criar dashboard TUI - Criar pseudo-janelas em texto - Adicionar tema escuro/claro
18 - Documentacao e apresentacao final	- Adicionar GIF - Criar release v1.0 - Adicionar roteiro de demo em video

16. Melhorias futuras e evolucao avancada

Interface textual avancada

- Dashboard com caixas e cores
- Menu inicial
- Pseudo-janelas em modo texto
- Tela de status do kernel
- Tema visual para apresentacao

Modo grafico experimental

- Framebuffer via GRUB/Multiboot
- gfx_putpixel
- gfx_draw_rect
- gfx_draw_string
- Tela grafica de boot
- Mini interface visual

Syscalls reais

- Interrupcao de syscall
- Tabela de chamadas
- Separacao conceitual entre shell e kernel
- Futuro modo usuario

Memoria avancada

- Mapa de memoria Multiboot
- Page allocator
- Paginacao
- Heap com free
- Protecao de memoria

Processos e escalonamento

- Tasks cooperativas
- Round-robin
- Context switch
- Comandos ps e kill
- Uptime real baseado no timer

Sistema de arquivos persistente

- Driver ATA/IDE
- Imagem de disco no QEMU
- FS simples persistente
- Comandos mkdir, rm, pwd
- ELF loader futuramente

Qualidade e produto

- CI para build
- Release com ISO
- GIF de demonstracao
- Manual do usuario
- Site ou GitHub Pages

17. Checklists finais

17.1 Checklist de ambiente

- WSL2 Ubuntu funcionando
- GCC, NASM, Make, LD e binutils instalados
- QEMU instalado
- GRUB e xorriso instalados
- GDB instalado
- Git configurado
- Repositorio clonado
- VS Code conectado ao WSL

17.2 Checklist de entrega minima

- Kernel carregado pelo GRUB
- boot.asm com Multiboot
- kernel_main em C
- linker.ld
- Makefile
- turingos.bin ou turingos.iso
- QEMU executando
- Mensagem na tela via VGA
- README completo
- docs/debugging.md
- docs/reverse-engineering.md
- Kanban atualizado
- Slides prontos

17.3 Checklist de apresentacao

- Abrir repositorio
- Mostrar Project/Kanban

- Mostrar roadmap
- Explicar arquitetura
- Rodar make run
- Mostrar QEMU
- Explicar codigo Assembly e C
- Mostrar evidencia de binario
- Mostrar engenharia reversa
- Mostrar melhorias futuras

18. Glossario

Termo	Explicacao
Boot	Processo inicial de carregamento do sistema
Bootloader	Programa que carrega o kernel; no projeto, GRUB
Kernel	Nucleo do sistema operacional
Assembly	Linguagem de baixo nivel proxima das instrucoes da CPU
Multiboot	Padrao usado pelo GRUB para reconhecer kernels
Linker	Ferramenta que junta objetos e gera binario final
VGA	Modo de video simples; em texto usa endereco 0xB8000
QEMU	Emulador/virtualizador usado para executar o TuringOS
GDB	Depurador usado para inspecionar execucao e registradores
Scancode	Codigo numerico enviado pelo teclado
IDT	Tabela de descritores de interrupcao
IRQ	Interrupcao gerada por hardware
PIT	Timer programavel usado para ticks
RAMFS	Sistema de arquivos temporario em memoria RAM
Framebuffer	Memoria grafica onde pixels podem ser desenhados

19. Referencias recomendadas

- OSDev Wiki - material pratico sobre boot, GRUB, VGA, IDT, teclado e memoria.
- The Little Book About OS Development - guia educacional de desenvolvimento de kernel.
- Intel Software Developer Manuals - referencia oficial da arquitetura x86.
- GNU GRUB Manual - funcionamento e configuracao do GRUB.
- QEMU Documentation - execucao e depuracao de maquinas virtuais.
- GDB Documentation - depuracao remota e inspecao de registradores.
- xv6: A Simple, Unix-like Teaching Operating System - referencia conceitual de sistema educacional.
- Minix - referencia historica e educacional de sistemas operacionais.
- Tanenbaum - Sistemas Operacionais Modernos.
- Silberschatz - Operating System Concepts.

Conclusao

O TuringOS e um projeto forte para Interface Hardware/Software porque demonstra, em um unico fluxo, boot, Assembly, C, geracao de binario, execucao virtualizada, comunicacao direta com hardware, drivers, depuracao, engenharia reversa e gestao de projeto com Git/Kanban.

A estrategia mais segura e entregar primeiro a base funcional: ambiente, boot, kernel minimo, Makefile, QEMU, VGA e terminal. A partir disso, o projeto pode crescer para shell, teclado, interrupcoes, timer, memoria, RAMFS, interface textual avancada e, futuramente, modo grafico experimental.

Mensagem final: Melhor entregar um mini kernel pequeno, funcional e bem documentado do que prometer um sistema operacional grande e incompleto. A qualidade da execucao, da demonstracao e da documentacao e o principal diferencial do TuringOS.